

EXPERIMENT NO.2 - WriteandexecuteSQLDataDefinitionLanguage

SQL> create table info(roll_no int, name varchar(10), class varchar(10), marks float);

Table created.

SQL> insert into info values('1','harsha','be','98.0');

1 row created.

SQL> insert into info values('2','Ram','be','90.0');

1 row created.

SQL> insert into info values('3','Shyam','be','85.0');

1 row created.

SQL> insert into info values('4','Seeta','TE','88.0');

1 row created.

SQL> select *from info;

ROLL_NO	NAME	CLASS	MARKS
1	harsha	be	98
2	Ram	be	90
3	Shyam	be	85
4	Seeta	TE	88

SQL> select roll_no,marks from info;

ROLL_NO	MARKS
1	98
2	90
3	85
4	88

SQL> delete from info where name='Shyam';

1 row deleted.

SQL> select *from info;

ROLL_NO	NAME	CLASS	MARKS
1	harsha	be	98
2	Ram	be	90
4	Seeta	TE	88

SQL> update info set class='SE' where roll_no='1';

1 row updated.

SQL> select *from info;

ROLL_NO	NAME	CLASS	MARKS
1	harsha	SE	98
2	Ram	be	90
4	Seeta	TE	88

SQL> alter table info add branch varchar(10);

Table altered.

SQL> select *from info;

ROLL_NO	NAME	CLASS	MARKS	BRANCH
1	harsha	SE	98	
2	Ram	be	90	
4	Seeta	TE	88	

SQL> desc info;

Name	Null?	Type
ROLL_NO		NUMBER(38)
NAME		VARCHAR2(10)
CLASS		VARCHAR2(10)
MARKS		FLOAT(126)
BRANCH		VARCHAR2(10)

SQL> alter table info drop column branch;

Table altered.

SQL> select *from info;

ROLL_NO	NAME	CLASS	MARKS
1	harsha	SE	98
2	Ram	be	90
4	Seeta	TE	88

SQL> truncate table info;

Table truncated.

SQL> select *from info;

no rows selected

SQL> desc info;

Name	Null?	Type
ROLL_NO		NUMBER(38)
NAME		VARCHAR2(10)
CLASS		VARCHAR2(10)
MARKS		FLOAT(126)

SQL> drop table info;

Table dropped.

EXPERIMENT NO.3

SQL Queries for Data Manipulation, Access Control, and Transactions

Design and run SQL queries to demonstrate the following:

- a) Data Manipulation (DML): Use SQL statements to INSERT, UPDATE, and DELETE records. Apply arithmetic, logical, set operators, pattern matching, and string functions.
- b) Access Control (DCL): Use GRANT, REVOKE, and ROLE commands to manage user access.
- c) Transaction Control (TCL): Apply START TRANSACTION, COMMIT, ROLLBACK, and SAVEPOINT commands to manage transactions.

```
SQL> CREATE TABLE Department (dept_id INT PRIMARY KEY, dept_name VARCHAR(20) UNIQUE NOT NULL, location VARCHAR(10));
```

Table created.

```
SQL> CREATE TABLE Student (student_id INT PRIMARY KEY, student_name VARCHAR(15) NOT NULL, email VARCHAR(20) UNIQUE, age INT CHECK (age >= 18), dept_id INT, admission_date DATE NOT NULL, FOREIGN KEY (dept_id) REFERENCES Department(dept_id));
```

Table created.

```
SQL> INSERT INTO Department VALUES (1, 'Computer', 'Pune');
```

1 row created.

```
SQL> INSERT INTO Department VALUES (2, 'Mechanical', 'Mumbai');
```

1 row created.

```
SQL> select * from Department;
```

DEPT_ID	DEPT_NAME	LOCATION
1	Computer	Pune
2	Mechanical	Mumbai

```
SQL> desc Department;
```

Name	Null?	Type
DEPT_ID	NOT NULL	NUMBER(38)
DEPT_NAME	NOT NULL	VARCHAR2(20)
LOCATION		VARCHAR2(10)

```
SQL> INSERT INTO Student VALUES (101, 'Amit', 'amit@gmail.com', 20, 1, '15 sep 2025');
```

1 row created.

```
SQL> INSERT INTO Student VALUES (102, 'Neha', 'neha@gmail.com', 19, 2, '16 aug 2024');
```

1 row created.

```
SQL> INSERT INTO Student VALUES (103, 'Rahul', 'rahul@gmail.com', 21, 1, '17 aug 2024');
```

1 row created.

SQL> select *from Student;

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	20	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24

SQL> UPDATE Student SET age = age + 1 WHERE student_id = 101;

1 row updated.

SQL> select *from Student;

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	21	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24

SQL> DELETE FROM Student WHERE age > 22 AND dept_id = 2;

0 rows deleted.

SQL> INSERT INTO Student VALUES (104, 'Sneha Joshi', 'sneha@gmail.com', 22, 1, '8 nov 2024');

1 row created.

SQL> DELETE FROM Student WHERE age > 22 AND dept_id = 2;

0 rows deleted.

SQL> DELETE FROM Student WHERE age > 21;

1 row deleted.

SQL> INSERT INTO Student VALUES (108, 'Sneha singh', 'singh@gmail.com', 22, 2, '8 dec 2024');

1 row created.

SQL> select *from Student;

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	21	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24
108	Sneha singh	singh@gmail.com	22	2	08-DEC-24

SQL> SELECT * FROM Student WHERE student_name LIKE 'R%';

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
103	Rahul	rahul@gmail.com	21	1	17-AUG-24

SQL> select *from Student;

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	21	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24
108	Sneha singh	singh@gmail.com	22	2	08-DEC-24

SQL> SELECT * FROM Student WHERE student_name LIKE 'N%';

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
102	Neha	neha@gmail.com	19	2	16-AUG-24

SQL> SELECT UPPER(student_name) AS Name_In_Caps, LENGTH(student_name) AS Name_Length FROM Student;

NAME_IN_CAPS	NAME_LENGTH
AMIT	4
NEHA	4
RAHUL	5
SNEHA SINGH	11

SQL> SELECT student_name FROM Student WHERE dept_id = 1 UNION SELECT student_name FROM Student WHERE age > 20;

STUDENT_NAME
Amit
Rahul
Sneha singh

SQL> GRANT SELECT, INSERT ON Student TO user1;
GRANT SELECT, INSERT ON Student TO user1
*

ERROR at line 1:
ORA-01917: user or role 'USER1' does not exist

SQL> commit;

Commit complete.

SQL> rollback;

Rollback complete.

```
SQL> INSERT INTO Student VALUES (105, 'Kiran More', 'kiran@gmail.com', 23, 2, '20 Dec2025');
```

1 row created.

```
SQL> select *from student;
```

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	21	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24
108	Sneha singh	singh@gmail.com	22	2	08-DEC-24
105	Kiran More	kiran@gmail.com	23	2	20-DEC-25

```
SQL> commit;
```

Commit complete.

```
SQL> DELETE FROM Student WHERE student_id = 105;
```

1 row deleted.

```
SQL> select *from student;
```

STUDENT_ID	STUDENT_NAME	EMAIL	AGE	DEPT_ID	ADMISSION
101	Amit	amit@gmail.com	21	1	15-SEP-25
102	Neha	neha@gmail.com	19	2	16-AUG-24
103	Rahul	rahul@gmail.com	21	1	17-AUG-24
108	Sneha singh	singh@gmail.com	22	2	08-DEC-24

EXPERIMENT NO.4

Use aggregate functions along with GROUPBY and HAVING clauses to retrieve summarized data from the database

```
SQL> CREATE TABLE Employee (EmpID NUMBER PRIMARY KEY, Name VARCHAR2(20), Department VARCHAR2(20), Salary NUMBER);
```

Table created.

```
SQL> INSERT INTO Employee VALUES (1, 'Amit', 'HR', 30000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (2, 'Neha', 'IT', 40000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (3, 'Raj', 'IT', 45000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (4, 'Priya', 'HR', 35000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (5, 'Kiran', 'Sales', 25000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (6, 'Rohit', 'Sales', 30000);
```

1 row created.

```
SQL> select *from employee;
```

EMPID	NAME	DEPARTMENT	SALARY
1	Amit	HR	30000
2	Neha	IT	40000
3	Raj	IT	45000
4	Priya	HR	35000
5	Kiran	Sales	25000
6	Rohit	Sales	30000

6 rows selected.

```
SQL> SELECT COUNT(*) AS Total_Employees FROM Employee;
```


TOTAL_EMPLOYEES

6

SQL> SELECT SUM(Salary) AS Total_Salary FROM Employee;

TOTAL_SALARY

205000

SQL> SELECT AVG(Salary) AS Average_Salary FROM Employee;

AVERAGE_SALARY

34166.6667

SQL> SELECT MIN(Salary) AS Minimum_Salary, MAX(Salary) AS Maximum_Salary FROM Employee;

MINIMUM_SALARY MAXIMUM_SALARY

25000 45000

SQL> SELECT Department, SUM(Salary) AS Total_Salary FROM Employee GROUP BY Department;

DEPARTMENT TOTAL_SALARY

Sales 55000
HR 65000
IT 85000

SQL> SELECT Department, COUNT(*) AS No_of_Employees, AVG(Salary) AS Avg_Salary, MAX(Salary) AS Highest_Salary FROM Employee GROUP BY Department;

DEPARTMENT NO_OF_EMPLOYEES AVG_SALARY HIGHEST_SALARY

Sales 2 27500 30000
HR 2 32500 35000
IT 2 42500 45000

SQL> SELECT Department, SUM(Salary) AS Total_Salary FROM Employee GROUP BY Department HAVING SUM(Salary) > 60000;

DEPARTMENT TOTAL_SALARY

HR 65000
IT 85000

EXPERIMENT NO.5

JOIN Operations and Views Perform various types of JOIN operations to extract meaningful relationships between tables. Create and manage different database views.

```
SQL> CREATE TABLE Department ( dept_id NUMBER PRIMARY KEY, dept_name VARCHAR2(10));
```

Table created.

```
SQL> INSERT INTO Department VALUES (1, 'HR');
```

1 row created.

```
SQL> INSERT INTO Department VALUES (2, 'IT');
```

1 row created.

```
SQL> INSERT INTO Department VALUES (3, 'Sales');
```

1 row created.

```
SQL> select * from Department;
```

```
DEPT_ID DEPT_NAME
-----
1 HR
2 IT
3 Sales
```

```
SQL> CREATE TABLE Employee ( emp_id NUMBER PRIMARY KEY, emp_name VARCHAR2(10), salary NUMBER,
dept_id NUMBER, FOREIGN KEY (dept_id) REFERENCES Department(dept_id) );
```

Table created.

```
SQL> INSERT INTO Employee VALUES (101, 'Amit', 30000, 1);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (102, 'Rahul', 40000, 2);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (103, 'Sneha', 35000, 2);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (104, 'Neha', 28000, NULL);
```

1 row created.

SQL> commit;

Commit complete.

SQL> select * from Employee;

EMP_ID	EMP_NAME	SALARY	DEPT_ID
101	Amit	30000	1
102	Rahul	40000	2
103	Sneha	35000	2
104	Neha	28000	

SQL> select * from Department;

DEPT_ID	DEPT_NAME
1	HR
2	IT
3	Sales

SQL> SELECT e.emp_name, e.salary, d.dept_name FROM Employee e INNER JOIN Department d ON e.dept_id = d.dept_id;

EMP_NAME	SALARY	DEPT_NAME
Amit	30000	HR
Rahul	40000	IT
Sneha	35000	IT

SQL> SELECT e.emp_name, d.dept_name FROM Employee e LEFT JOIN Department d ON e.dept_id = d.dept_id;

EMP_NAME	DEPT_NAME
Amit	HR
Rahul	IT
Sneha	IT
Neha	

SQL> SELECT e.emp_name, d.dept_name FROM Employee e RIGHT JOIN Department d ON e.dept_id = d.dept_id;

EMP_NAME	DEPT_NAME
Amit	HR
Rahul	IT
Sneha	IT
	Sales

```
SQL> SELECT e.emp_name, d.dept_name FROM Employee e FULL OUTER JOIN Department d ON e.dept_id = d.dept_id;
```

```
EMP_NAME  DEPT_NAME
```

```
-----
```

```
Amit      HR
```

```
Rahul     IT
```

```
Sneha     IT
```

```
Neha
```

```
      Sales
```

```
SQL> CREATE VIEW emp_view AS SELECT emp_name, salary FROM Employee;
```

View created.

```
SQL> SELECT * FROM emp_view;
```

```
EMP_NAME  SALARY
```

```
-----
```

```
Amit      30000
```

```
Rahul     40000
```

```
Sneha     35000
```

```
Neha      28000
```

```
SQL> CREATE VIEW emp_dept_view AS SELECT e.emp_name, e.salary, d.dept_name FROM Employee e JOIN Department d ON e.dept_id = d.dept_id;
```

View created.

EXPERIMENT NO.6

Write and execute subquerie store trieve data from one tablebase don results from another.

```
SQL> CREATE TABLE Department(Dept_ID NUMBER PRIMARY KEY, Dept_Name VARCHAR(10));
```

Table created.

```
SQL> INSERT INTO Department VALUES (1, 'HR');
```

1 row created.

```
SQL> INSERT INTO Department VALUES (2, 'IT');
```

1 row created.

```
SQL> INSERT INTO Department VALUES (3, 'Finance');
```

1 row created.

```
SQL> select * from Department;
```

```
DEPT_ID DEPT_NAME
-----
1 HR
2 IT
3 Finance
```

```
SQL> CREATE TABLE Employee(Emp_ID NUMBER PRIMARY KEY, Emp_Name VARCHAR(10), Salary NUMBER,
Dept_ID NUMBER, FOREIGN KEY(Dept_ID) REFERENCES Department(Dept_ID));
```

Table created.

```
SQL> INSERT INTO Employee VALUES (101, 'Amit', 40000, 2);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (102, 'Neha', 35000, 1);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (103, 'Rahul', 50000, 2);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (104, 'Priya', 30000, 3);
```

1 row created.

```
SQL> select * from Employee;
```

EMP_ID	EMP_NAME	SALARY	DEPT_ID
101	Amit	40000	2
102	Neha	35000	1
103	Rahul	50000	2
104	Priya	30000	3

```
SQL> select * from Department;
```

DEPT_ID	DEPT_NAME
1	HR
2	IT
3	Finance

```
SQL> SELECT Emp_Name From Employee WHERE Dept_ID = (SELECT Dept_ID FROM Department WHERE Dept_Name = 'IT');
```

EMP_NAME
Amit
Rahul

```
SQL> SELECT Emp_Name, Salary From Employee WHERE Salary > (SELECT AVG(Salary) FROM Employee);
```

EMP_NAME	SALARY
Amit	40000
Rahul	50000

```
SQL> SELECT Emp_Name From Employee WHERE Dept_ID IN (SELECT Dept_ID FROM Department WHERE Dept_Name IN ('HR', 'Finance'));
```

```
EMP_NAME
```

```
-----
```

```
Neha
```

```
Priya
```

```
SQL> SELECT AVG(Salary) AS Avg_Salary FROM (SELECT Salary FROM Employee);
```

```
AVG_SALARY
```

```
-----
```

```
38750
```

```
SQL> SELECT Emp_Name, Salary From Employee E WHERE Salary > (SELECT AVG(Salary) FROM Employee WHERE Dept_ID = E.Dept_ID);
```

```
EMP_NAME    SALARY
```

```
-----
```

```
Rahul      50000
```

EXPERIMENT NO.7

Stored Procedures or Function with Cursors Create and execute stored procedures/function using cursors.

```
SQL> SET SERVEROUTPUT ON;
```

```
SQL> CREATE TABLE employee ( emp_id NUMBER, emp_name VARCHAR2(50), salary NUMBER );
```

Table created.

```
SQL> INSERT INTO Employee VALUES (101, 'Amit', 40000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (102, 'prasad', 50000);
```

1 row created.

```
SQL> INSERT INTO Employee VALUES (103, 'ram', 60000);
```

1 row created.

```
SQL> SELECT * FROM Employee;
```

EMP_ID	EMP_NAME	SALARY
101	Amit	40000
102	prasad	50000
103	ram	60000

```
SQL> CREATE OR REPLACE PROCEDURE emp_cursor_proc
```

```
2 IS
```

```
3 CURSOR emp_cursor IS
```

```
4 SELECT emp_id, emp_name, salary FROM employee;
```

```
5 v_id employee.emp_id%TYPE;
```

```
6 v_name employee.emp_name%TYPE;
```

```
7 v_salary employee.salary%TYPE;
```

```
8 BEGIN
```

```
9 OPEN emp_cursor;
```

```
10 LOOP
```



```
11 FETCH emp_cursor INTO v_id, v_name, v_salary;
12 EXIT WHEN emp_cursor%NOTFOUND;
13 DBMS_OUTPUT.PUT_LINE('ID: ' || v_id ||
14 ' Name: ' || v_name ||
15 ' Salary: ' || v_salary);
16 END LOOP;
17 CLOSE emp_cursor;
18 END;
19 /
```

Procedure created.

SQL> BEGIN

```
2 emp_cursor_proc;
3 END;
4 /
```

ID: 101 Name: Amit Salary: 40000

ID: 102 Name: prasad Salary: 50000

ID: 103 Name: ram Salary: 60000

PL/SQL procedure successfully completed.

SQL> CREATE OR REPLACE FUNCTION total_salary

```
2 RETURN NUMBER
3 IS
4 CURSOR sal_cursor IS
5 SELECT salary FROM employee;
6 v_salary employee.salary%TYPE;
7 total NUMBER := 0;
8 BEGIN
9 OPEN sal_cursor;
10 LOOP
11 FETCH sal_cursor INTO v_salary;
12 EXIT WHEN sal_cursor%NOTFOUND;
```

```
13 total := total + v_salary;
14 END LOOP;
15 CLOSE sal_cursor;
16 RETURN total;
17 END;
18 /
```

Function created.

SQL> DECLARE

```
2 result NUMBER;
3 BEGIN
4 result := total_salary;
5 DBMS_OUTPUT.PUT_LINE('Total Salary: ' || result);
6 END;
7 /
```

Total Salary: 150000

PL/SQL procedure successfully completed.

EXPERIMENT NO.8

Database Triggers Implement and test triggers to maintain data integrity in database.

```
SQL> CREATE TABLE employees ( emp_id NUMBER, name VARCHAR2(50), salary NUMBER);
```

Table created.

```
SQL> INSERT INTO Employees VALUES (101, 'Amit', 40000);
```

1 row created.

```
SQL> INSERT INTO Employees VALUES (102, 'prasad', 50000);
```

1 row created.

```
SQL> INSERT INTO Employees VALUES (103, 'ram', 60000);
```

1 row created.

```
SQL> CREATE OR REPLACE TRIGGER trg_salary_check
```

```
2 BEFORE INSERT ON employees
```

```
3 FOR EACH ROW
```

```
4 BEGIN
```

```
5 IF :NEW.salary < 10000 THEN
```

```
6 :NEW.salary := 10000;
```

```
7 END IF;
```

```
8 END;
```

```
9 /
```

Trigger created.

```
SQL> INSERT INTO employees VALUES (1, 'Amit', 5000);
```

1 row created.

```
SQL> Select * from employees;
```

EMP_ID NAME	SALARY
101 Amit	40000
102 prasad	50000
103 ram	60000
1 Amit	10000

```
SQL> CREATE OR REPLACE TRIGGER trg_after_insert
2 AFTER INSERT ON employees
3 FOR EACH ROW
4 BEGIN
5 DBMS_OUTPUT.PUT_LINE('Record inserted successfully');
6 END;
7 /
```

Trigger created.

```
SQL> CREATE OR REPLACE TRIGGER trg_salary_update
2 BEFORE UPDATE ON employees
3 FOR EACH ROW
4 BEGIN
5 IF :NEW.salary < :OLD.salary THEN
6 RAISE_APPLICATION_ERROR(-20001, 'Salary cannot be reduced');
7 END IF;
8 END;
9 /
```

Trigger created.

```
SQL> CREATE OR REPLACE TRIGGER trg_no_delete
2 BEFORE DELETE ON employees
3 BEGIN
```

```
4 RAISE_APPLICATION_ERROR(-20002, 'Deletion not allowed');  
5 END;  
6 /
```

Trigger created.

```
SQL> INSERT INTO employees VALUES (2, 'shyam', 8000);  
Record inserted successfully
```

```
SQL> UPDATE employees SET salary = 8000 WHERE emp_id = 1;  
UPDATE employees SET salary = 8000 WHERE emp_id = 1  
*
```

ERROR at line 1:

ORA-20001: Salary cannot be reduced

ORA-06512: at "SYSTEM.TRG_SALARY_UPDATE", line 3

ORA-04088: error during execution of trigger 'SYSTEM.TRG_SALARY_UPDATE'

```
SQL> DELETE FROM employees WHERE emp_id = 1;  
DELETE FROM employees WHERE emp_id = 1  
*
```

ERROR at line 1:

ORA-20002: Deletion not allowed

ORA-06512: at "SYSTEM.TRG_NO_DELETE", line 2

ORA-04088: error during execution of trigger 'SYSTEM.TRG_NO_DELETE'

EXPERIMENT NO.9

CRUD Operations using MongoDB Design and implement basic Create ,Read, Update, and Delete(CRUD) operations using MongoDB. Use the save method and logical operators where necessary

```
localhost> use collegeDB switched to db collegeDB
```

```
collegeDB> db.createCollection("students")
{ ok: 1 }
```

```
collegeDB> db.students.insertOne({
|  name: "Amit",
|  age: 21,
|  course: "CSE",
|  marks: 82
| });

{
  acknowledged: true,
  insertedId: ObjectId('69dcae0cf0836db3563682d1')
}
```

```
collegeDB> db.students.insertMany([
|  { name: "Riya", age: 22, course: "IT", marks: 75 },
|  { name: "Sohan", age: 23, course: "Mechanical", marks: 65 },
|  { name: "Meena", age: 21, course: "CSE", marks: 90 }
| ]);

{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('69dcae18f0836db3563682d2'),
    '1': ObjectId('69dcae18f0836db3563682d3'),
    '2': ObjectId('69dcae18f0836db3563682d4')
  }
}
```

```
collegeDB> db.students.find()
[
  {
    _id: ObjectId('69dcae0cf0836db3563682d1'),
    name: 'Amit',
    age: 21,
    course: 'CSE',
    marks: 82
  },
]
```

```

{
  _id: ObjectId('69dcae18f0836db3563682d2'),
  name: 'Riya',
  age: 22,
  course: 'IT',
  marks: 75
},
{
  _id: ObjectId('69dcae18f0836db3563682d3'),
  name: 'Sohan',
  age: 23,
  course: 'Mechanical',
  marks: 65
},
{
  _id: ObjectId('69dcae18f0836db3563682d4'),
  name: 'Meena',
  age: 21,
  course: 'CSE',
  marks: 90
}
]

```

```
collegeDB> db.students.find({}, { name: 1, marks: 1 })
```

```

[
  { _id: ObjectId('69dcae0cf0836db3563682d1'), name: 'Amit', marks: 82 },
  { _id: ObjectId('69dcae18f0836db3563682d2'), name: 'Riya', marks: 75 },
  { _id: ObjectId('69dcae18f0836db3563682d3'), name: 'Sohan', marks: 65 },
  { _id: ObjectId('69dcae18f0836db3563682d4'), name: 'Meena', marks: 90 }
]

```

```
collegeDB> db.students.find({ course: "CSE" })
```

```

[
  {
    _id: ObjectId('69dcae0cf0836db3563682d1'),
    name: 'Amit',
    age: 21,
    course: 'CSE',
    marks: 82
  },
  {
    _id: ObjectId('69dcae18f0836db3563682d4'),
    name: 'Meena',
    age: 21,
    course: 'CSE',
    marks: 90
  }
]

```

```

collegeDB> db.students.updateOne(
|   { name: "Amit" },
|   { $set: { marks: 88 } }
| )

```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
```

```
collegeDB> db.students.updateMany(
| { course: "CSE" },
| { $set: { course: "Computer Science" } }
| )
```

```
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 2,
  modifiedCount: 2,
  upsertedCount: 0
}
```

```
collegeDB> db.students.deleteOne({ name: "Sohan" })
{ acknowledged: true, deletedCount: 1 }
collegeDB> db.students.deleteMany({ marks: { $lt: 70 } })
{ acknowledged: true, deletedCount: 0 }
collegeDB>
```


EXPERIMENT NO.10

Aggregation and Indexing in MongoDB Design and execute Mongo DB queries using aggregation and indexing techniques with suitable example

```
localhost> use collegeDB
```

```
switched to db collegeDB
```

```
collegeDB> db.students.insertMany([
| { _id: 1, name: "Amit", age: 21, department: "Computer", marks: 85, city: "Pune" },
| { _id: 2, name: "Neha", age: 22, department: "IT", marks: 90, city: "Mumbai" },
| { _id: 3, name: "Ravi", age: 21, department: "Computer", marks: 75, city: "Pune" },
| { _id: 4, name: "Priya", age: 23, department: "Mechanical", marks: 60, city: "Delhi" },
| { _id: 5, name: "Anjali", age: 20, department: "Computer", marks: 95, city: "Pune" }
| ])
{
  acknowledged: true,
  insertedIds: { '0': 1, '1': 2, '2': 3, '3': 4, '4': 5 }
}
```

```
collegeDB> db.students.aggregate([
| { $group: { _id: "$department", avgMarks: { $avg: "$marks" } } }
| ])
[
  { _id: 'IT', avgMarks: 90 },
  { _id: 'Mechanical', avgMarks: 60 },
  { _id: 'Computer', avgMarks: 85 }
]
```

```
collegeDB> db.students.aggregate([
| { $group: { _id: "$city", totalStudents: { $sum: 1 } } }
| ])
[
  { _id: 'Mumbai', totalStudents: 1 },
  { _id: 'Pune', totalStudents: 3 },
  { _id: 'Delhi', totalStudents: 1 }
]
```

```
collegeDB> db.students.aggregate([
| { $match: { marks: { $gt: 80 } } },
| { $group: { _id: "$department", highScorers: { $sum: 1 } } }
| ])
[ { _id: 'IT', highScorers: 1 }, { _id: 'Computer', highScorers: 2 } ]
```

```
collegeDB> db.students.aggregate([
| { $sort: { marks: -1 } }
| ])
```

collegeDB>

```
{
  _id: 5,
  name: 'Anjali',
  age: 20,
  department: 'Computer',
  marks: 95,
  city: 'Pune'
},
{
  _id: 2,
  name: 'Neha',
  age: 22,
  department: 'IT',
  marks: 90,
  city: 'Mumbai'
},
{
  _id: 1,
  name: 'Amit',
  age: 21,
  department: 'Computer',
  marks: 85,
  city: 'Pune'
},
{
  _id: 3,
  name: 'Ravi',
  age: 21,
  department: 'Computer',
  marks: 75,
  city: 'Pune'
},
{
  _id: 4,
  name: 'Priya',
  age: 23,
  department: 'Mechanical',
  marks: 60,
  city: 'Delhi'
}
]
```

collegeDB> db.students.createIndex({ department: 1 })
department_1

collegeDB> db.students.find({ department: "Computer" })

```
[
  {
    _id: 1,
    name: 'Amit',
    age: 21,
```

```
department: 'Computer',
marks: 85,
city: 'Pune'
},
{
  _id: 3,
  name: 'Ravi',
  age: 21,
  department: 'Computer',
  marks: 75,
  city: 'Pune'
},
{
  _id: 5,
  name: 'Anjali',
  age: 20,
  department: 'Computer',
  marks: 95,
  city: 'Pune'
}
]
```

```
collegeDB> db.students.createIndex({ city: 1, marks: -1 })
city_1_marks_-1
```

```
collegeDB> db.students.find({ city: "Pune" }).sort({ marks: -1 })
[
  {
    _id: 5,
    name: 'Anjali',
    age: 20,
    department: 'Computer',
    marks: 95,
    city: 'Pune'
  },
  {
    _id: 1,
    name: 'Amit',
    age: 21,
    department: 'Computer',
    marks: 85,
    city: 'Pune'
  },
  {
    _id: 3,
    name: 'Ravi',
    age: 21,
    department: 'Computer',
    marks: 75,
    city: 'Pune'
  }
]
```

```
collegeDB> db.students.aggregate([
| { $match: { city: "Pune", marks: { $gt: 70 } } },
| { $sort: { marks: -1 } }ame: '_id_' },
| ])v: 2, key: { department: 1 }, name: 'department_1' },
[ { v: 2, key: { city: 1, marks: -1 }, name: 'city_1_marks_-1' }
{
  _id: 5,
  name: 'Anjali',
  age: 20,
  department: 'Computer',
  marks: 95,
  city: 'Pune'
},
{
  _id: 1,
  name: 'Amit',
  age: 21,
  department: 'Computer',
  marks: 85,
  city: 'Pune'
},
{
  _id: 3,
  name: 'Ravi',
  age: 21,
  department: 'Computer',
  marks: 75,
  city: 'Pune'
}
]
```

```
collegeDB> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 5 }
```

```
collegeDB> db.students.drop()
true
```

```
collegeDB> db.dropDatabase()
{ ok: 1, dropped: 'collegeDB' }
```